

Documentação do Simulador de Sistemas Operacionais

Gerenciamento de Processos e Memória

Michel Rochytor Lima Barbosa

Semestre Letivo – 2026

1 Introdução

Este documento apresenta a documentação técnica e funcional de um simulador de Sistemas Operacionais desenvolvido em linguagem C, com interface gráfica baseada na biblioteca Kosmos. O sistema tem como objetivo demonstrar, de forma visual e prática, o funcionamento de dois pilares fundamentais de um sistema operacional: o escalonamento de processos e o gerenciamento de memória virtual por paginação.

O simulador permite carregar uma lista de processos a partir de um arquivo CSV, configurar parâmetros de memória, escolher algoritmos de escalonamento e selecionar políticas de substituição de páginas. Ao final da simulação, o programa exibe uma linha do tempo em formato de gráfico de Gantt, logs de execução, métricas de desempenho e a quantidade total de page faults ocorridos.

O código-fonte do projeto está disponível em: <https://github.com/MichelRochytor/SimuladorS.O.git>. O instalador do framework Kosmos pode ser obtido em: <https://github.com/MichelRochytor/KosmosUI-Installer/releases/tag/Instalador>.

2 Objetivos do Projeto

2.1 Objetivo Geral

Desenvolver um simulador integrado de Sistemas Operacionais capaz de representar o comportamento de processos em execução e a ocupação da memória física durante a aplicação de políticas de escalonamento e substituição de páginas.

2.2 Objetivos Específicos

- Implementar uma interface gráfica funcional para entrada de dados e visualização dos resultados.

- Ler processos a partir de um arquivo CSV contendo tempo de chegada, burst de CPU, prioridade e memória necessária.
- Simular os algoritmos de escalonamento Round-Robin, SJF Preemptivo e Prioridade Preemptiva.
- Simular memória virtual utilizando paginação com tamanho fixo de página.
- Implementar políticas de substituição de páginas FIFO, LRU e uma política aleatória associada à opção de algoritmo ótimo na interface.
- Exibir métricas de desempenho, como tempo médio de resposta, tempo médio de espera e total de page faults.

3 Tecnologias Utilizadas

O projeto foi implementado em linguagem C, utilizando a biblioteca Kosmos para construção da interface gráfica. A Tabela 1 resume os principais recursos utilizados.

Recurso	Descrição
Linguagem	C
Interface gráfica	Biblioteca Kosmos
Entrada de dados	Arquivo CSV
Visualização	ListView, ComboBox, RichEdit e campos estáticos
Memória	Alocação dinâmica com <code>realloc</code> para representar os frames da RAM
Sistema alvo	Windows e Linux

Table 1: Tecnologias e recursos utilizados no simulador.

4 Requisitos do Sistema

4.1 Entrada de Dados

O simulador recebe como entrada um arquivo CSV contendo uma lista de processos. Cada linha representa um processo e deve conter os seguintes campos:

1. Tempo de chegada;
2. Tempo de execução, também chamado de CPU Burst;
3. Prioridade;
4. Memória necessária em MB.

O arquivo pode utilizar vírgula ou ponto e vírgula como separador. O código também aceita a presença de um cabeçalho contendo termos como **Tempo**, **id** ou **Chegada**. Quando o cabeçalho não é identificado, a leitura retorna ao início do arquivo para processar a primeira linha como dado válido.

Um exemplo de arquivo CSV aceito é apresentado a seguir:

```
Tempo de Chegada;Burst CPU;Prioridade;Memoria
0;5;2;12
1;3;4;8
2;7;1;20
4;2;3;4
```

Durante o carregamento, os processos recebem identificadores automáticos no formato P1, P2, P3 e assim por diante.

4.2 Interface Gráfica

A interface gráfica permite que o usuário interaja com o simulador sem utilizar terminal. Os principais elementos da interface são:

- Botão para procurar e carregar o arquivo CSV de processos.
- Campo para exibir o caminho do arquivo selecionado.
- Lista de processos carregados, contendo ID, tempo de chegada, burst, prioridade e memória requisitada.
- Campo para configurar o tamanho da memória física.
- Campo para configurar o tamanho da memória virtual.
- Campo para configurar o quantum do Round-Robin.
- ComboBox para selecionar o algoritmo de escalonamento.
- ComboBox para selecionar a política de substituição de páginas.
- Área de log para acompanhar a execução e os page faults.
- Área RichEdit para exibição colorida do gráfico de Gantt.
- Campos de saída para tempo médio de resposta, tempo médio de espera e total de page faults.

5 Estruturas de Dados

O simulador utiliza três estruturas principais para representar processos, memória e histórico de execução.

5.1 Estrutura de Processo

A estrutura `Processo` armazena todas as informações necessárias para a simulação de escalonamento e cálculo das métricas.

Campo	Descrição
<code>id</code>	Identificador do processo, como P1 ou P2.
<code>tempo_chegada</code>	Instante em que o processo entra no sistema.
<code>burst_total</code>	Tempo total de CPU necessário para concluir o processo.
<code>burst_restante</code>	Tempo de CPU ainda pendente durante a simulação.
<code>prioridade</code>	Valor usado no escalonamento por prioridade.
<code>memoria_mb</code>	Quantidade de memória solicitada pelo processo.
<code>paginas_necessarias</code>	Número de páginas calculado a partir da memória solicitada.
<code>tempo_inicio</code>	Primeiro instante em que o processo executa.
<code>tempo_fim</code>	Instante de conclusão do processo.
<code>tempo_espera</code>	Tempo total de espera calculado ao final.
<code>tempo_resposta</code>	Diferença entre o primeiro atendimento e a chegada.
<code>iniciado</code>	Indica se o processo já iniciou execução.
<code>concluido</code>	Indica se o processo foi finalizado.

Table 2: Campos da estrutura `Processo`.

5.2 Estrutura de Frame de Memória

A estrutura `FrameMemoria` representa um quadro da memória física.

Campo	Descrição
<code>numero_pagina</code>	Número da página armazenada no frame.
<code>proc_id</code>	Identificador do processo dono da página.
<code>ultimo_acesso</code>	Marcador usado pela política LRU para identificar a página menos recentemente utilizada.

Table 3: Campos da estrutura `FrameMemoria`.

5.3 Estrutura de Histórico de Execução

A estrutura `RegistroExecucao` registra qual processo executou em cada instante da simulação. Essa informação é utilizada posteriormente para renderizar o gráfico de Gantt.

6 Funcionamento Geral do Simulador

O funcionamento do simulador pode ser dividido em cinco etapas principais:

Campo	Descrição
instante	Unidade de tempo registrada.
proc_id	Processo executado no instante. Quando vazio, representa CPU ociosa.

Table 4: Campos da estrutura `RegistroExecucao`.

1. O usuário seleciona um arquivo CSV contendo os processos.
2. O programa carrega os processos, calcula o número de páginas necessárias e exibe os dados na lista da interface.
3. O usuário define memória física, memória virtual, quantum, algoritmo de escalonamento e política de substituição.
4. Ao iniciar a simulação, o sistema executa os processos unidade por unidade ou por quantum, conforme o algoritmo selecionado.
5. Ao final, a interface apresenta logs, gráfico de Gantt e métricas de desempenho.

7 Fluxogramas do Código

Os fluxogramas a seguir resumem o funcionamento interno do simulador, desde a inicialização da interface até a execução dos algoritmos de escalonamento e gerenciamento de memória.

7.1 Fluxo Geral da Aplicação

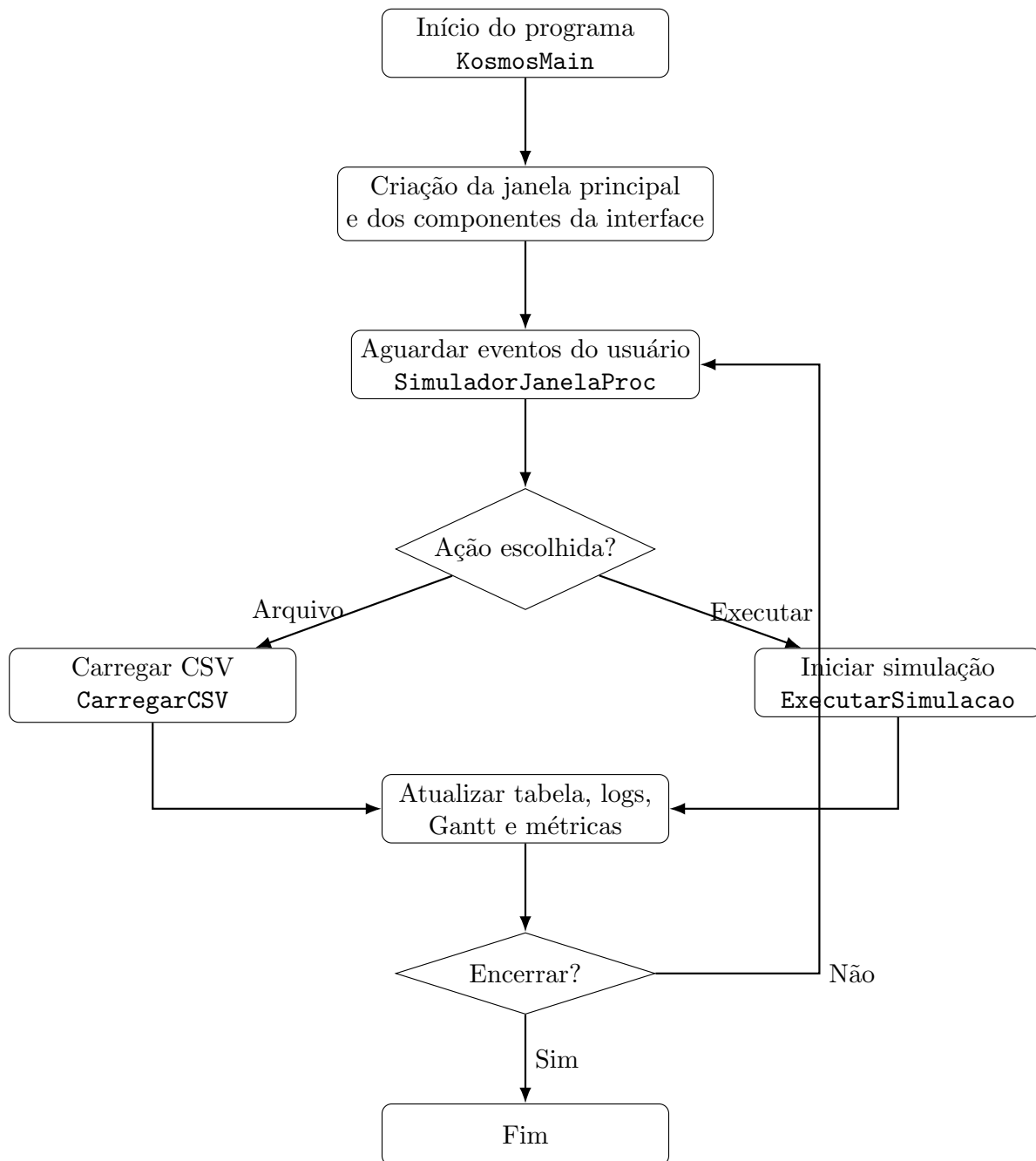


Figure 1: Fluxo geral da aplicação e tratamento de eventos da interface.

7.2 Fluxo de Carregamento do CSV

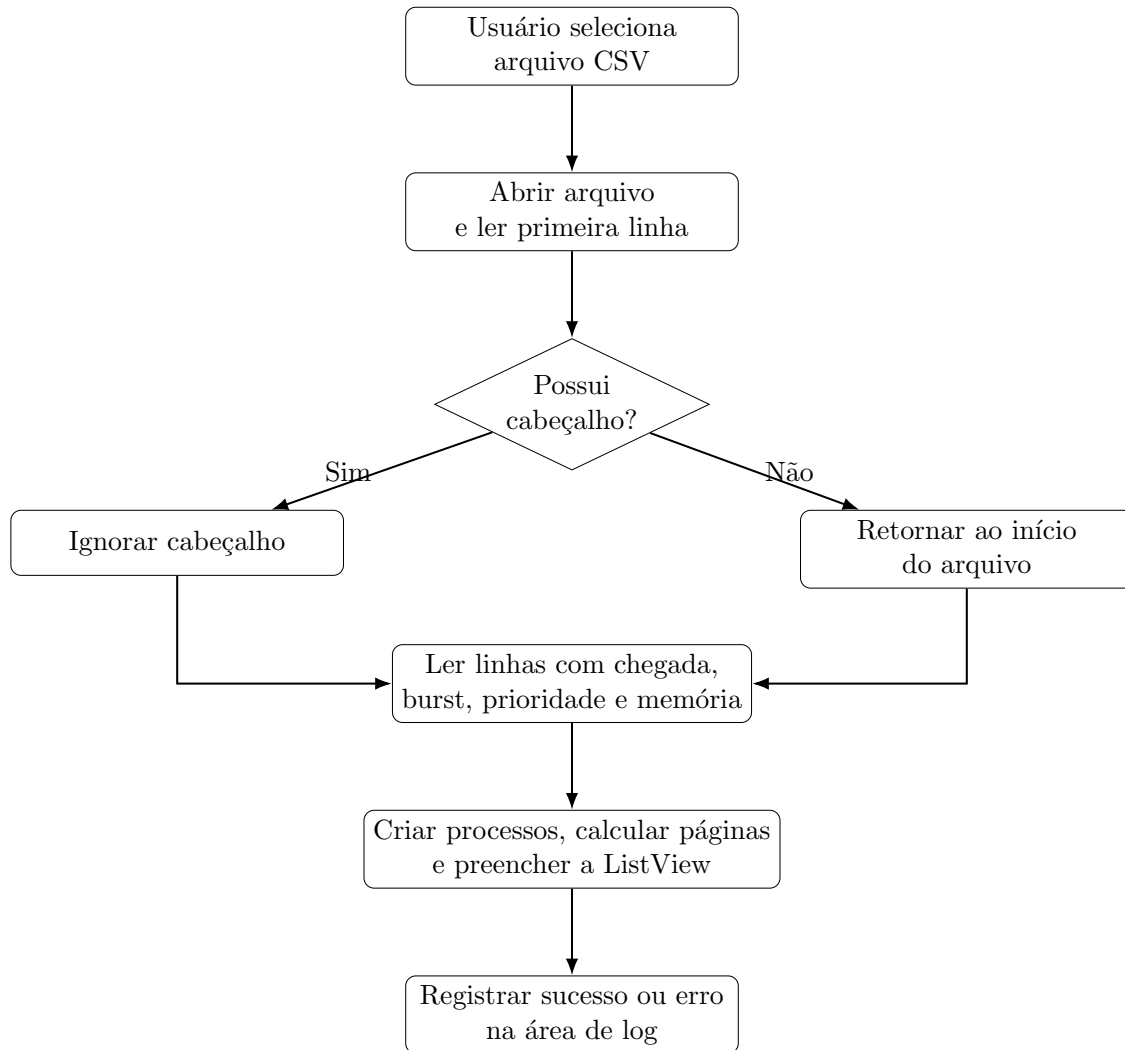


Figure 2: Fluxo de leitura do arquivo CSV e criação dos processos.

7.3 Fluxo da Simulação e Memória

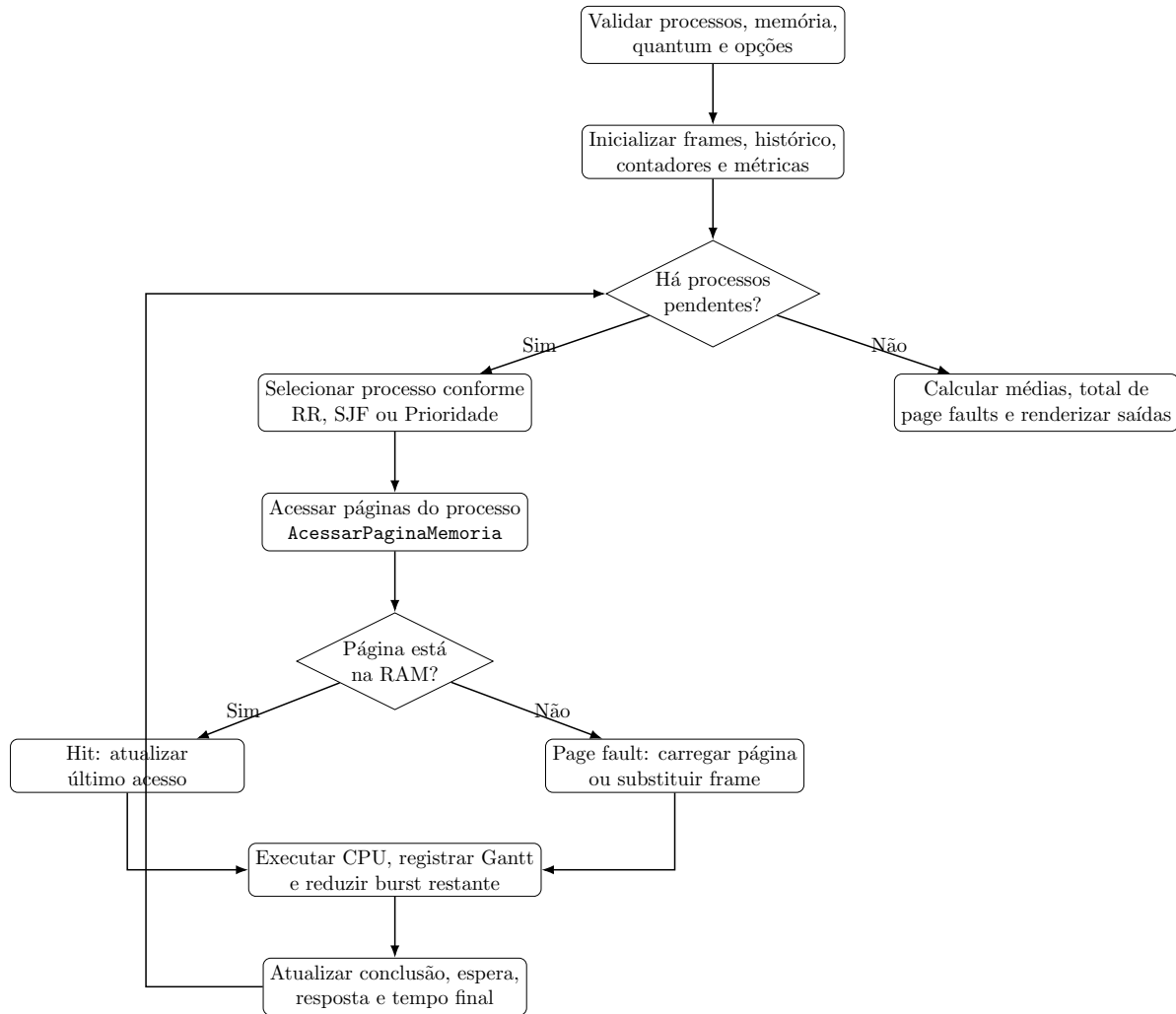


Figure 3: Fluxo principal de execução da simulação e do gerenciamento de páginas.

8 Escalonamento de Processos

O núcleo da simulação escolhe, a cada ciclo, qual processo deve ocupar a CPU. A escolha depende do algoritmo selecionado na interface.

8.1 Round-Robin

No Round-Robin, os processos prontos são executados em ordem circular. Cada processo recebe uma fatia de tempo chamada *quantum*. Quando o *quantum* termina e o processo ainda possui *burst* restante, ele volta para a fila circular e aguarda uma nova oportunidade de execução.

No código, a variável `index_rr` mantém a posição de busca do próximo processo. O número de unidades de tempo executadas é definido por:

$$passos = \min(quantum, burst_restante) \quad (1)$$

Esse algoritmo evita que um único processo monopolize a CPU, sendo adequado para sistemas interativos.

8.2 SJF Preemptivo

O SJF Preemptivo, também conhecido como Shortest Remaining Time First, seleciona o processo pronto com menor tempo restante de execução. A cada unidade de tempo, o simulador reavalia os processos disponíveis e escolhe aquele com o menor *burst_restante*.

Essa estratégia tende a reduzir o tempo médio de espera, porém pode causar inanição de processos longos caso processos curtos cheguem continuamente.

8.3 Prioridade Preemptiva

No escalonamento por Prioridade Preemptiva, o simulador escolhe o processo pronto com maior valor de prioridade. Assim como no SJF Preemptivo, a execução ocorre em passos de uma unidade de tempo, permitindo que um processo com maior prioridade assuma a CPU assim que estiver disponível.

Neste projeto, quanto maior o número da prioridade, maior é a prioridade do processo.

9 Gerenciamento de Memória

O simulador implementa memória virtual com paginação. A página possui tamanho fixo de 4 MB, definido pela constante `TAM_PAGINA_MB`. O número de páginas necessárias para cada processo é

calculado por:

$$paginas_necessarias = \left\lceil \frac{memoria_mb}{4} \right\rceil \quad (2)$$

No código, esse cálculo é realizado com aritmética inteira:

```
(memoria_mb + TAM_PAGINA_MB - 1) / TAM_PAGINA_MB
```

A memória física é dividida em frames do mesmo tamanho das páginas. O total de frames da RAM é calculado por:

$$frames = \frac{memoria_fisica}{4} \quad (3)$$

Caso o valor configurado para memória física seja menor que 4 MB, o simulador garante ao menos um frame.

9.1 Acesso às Páginas

Durante a execução de um processo, o simulador tenta acessar todas as páginas necessárias para aquele processo. O acesso pode resultar em:

- **Hit:** a página já está presente na memória física.
- **Page fault:** a página não está presente e precisa ser carregada para algum frame.

Quando ocorre um page fault, o contador total de falhas de página é incrementado. Se houver frame livre, a página é carregada diretamente. Caso a memória esteja cheia, uma política de substituição escolhe qual frame será removido.

9.2 Política FIFO

A política FIFO, First-In, First-Out, substitui a página mais antiga seguindo uma ordem circular de frames. O simulador utiliza a variável global `ponteiro_fifo` para apontar o próximo frame que será substituído.

Após cada substituição, o ponteiro avança para o próximo frame:

$$ponteiro_fifo = (ponteiro_fifo + 1) \bmod total_frames_ram \quad (4)$$

9.3 Política LRU

A política LRU, Least Recently Used, substitui a página menos recentemente acessada. Para isso, o simulador mantém um contador global chamado `contador_acessos_memoria`. A cada tentativa de acesso, esse contador é incrementado.

Quando uma página é acessada com sucesso, seu campo `ultimo_acesso` é atualizado. Em caso de substituição, o frame com menor valor de `ultimo_acesso` é escolhido.

9.4 Política Aleatória / Opção Ótimo

Na interface, a terceira opção aparece como `Algoritmo Otimo`. Entretanto, na implementação atual, essa opção realiza uma substituição aleatória utilizando:

```
rand() % total_frames_ram
```

Portanto, embora a interface apresente a opção como algoritmo ótimo, o comportamento implementado corresponde a uma política aleatória. Para implementar o algoritmo ótimo real, seria necessário conhecer previamente a sequência futura de acessos às páginas e substituir a página cujo próximo uso estivesse mais distante ou que não fosse mais utilizada.

10 Relatórios e Saídas

Ao finalizar a simulação, o programa apresenta os resultados diretamente na interface gráfica.

10.1 Log de Execução

A área de log exibe mensagens sobre o andamento da simulação, incluindo:

- início da simulação;
- processo executado em cada instante ou intervalo;
- quantidade de unidades de tempo executadas;
- ocorrências de page faults;
- mensagens de erro ou sucesso no carregamento do CSV.

10.2 Gráfico de Gantt

O gráfico de Gantt é renderizado em um componente RichEdit utilizando caracteres Unicode e cores diferentes para cada processo. A visualização possui:

- régua de tempo na primeira linha;
- uma linha para cada processo carregado;
- blocos coloridos indicando os instantes em que cada processo executou;
- linha específica para representar períodos de CPU ociosa.

Essa representação facilita a análise visual da ordem de execução dos processos e dos efeitos dos algoritmos preemptivos.

10.3 Métricas de Performance

O simulador calcula três métricas principais ao final da execução.

10.3.1 Tempo Médio de Resposta

O tempo de resposta de um processo é o intervalo entre seu tempo de chegada e o primeiro instante em que ele recebe a CPU:

$$tempo_resposta = tempo_inicio - tempo_chegada \quad (5)$$

O tempo médio de resposta é calculado por:

$$TMR = \frac{\sum tempo_resposta}{n} \quad (6)$$

10.3.2 Tempo Médio de Espera

O tempo de espera considera o tempo total em que o processo permaneceu no sistema sem executar:

$$tempo_espera = (tempo_fim - tempo_chegada) - burst_total \quad (7)$$

O tempo médio de espera é calculado por:

$$TME = \frac{\sum tempo_espera}{n} \quad (8)$$

10.3.3 Total de Page Faults

O total de page faults corresponde à soma de todas as falhas de página ocorridas durante a simulação. Esse valor permite comparar o impacto das políticas de substituição na memória física

configurada.

11 Descrição das Principais Funções

Função	Responsabilidade
GanttAppendColorido	Adiciona texto colorido ao RichEdit usado para exibir o gráfico de Gantt.
AdicionarLog	Insere mensagens coloridas na área de log da interface.
RenderizarGantt	Monta a régua de tempo, as linhas dos processos e a linha de ociosidade.
AcessarPaginaMemoria	Verifica hits e page faults, carrega páginas e aplica substituição quando necessário.
CarregarCSV	Lê o arquivo CSV, cria os processos e preenche a ListView.
ExecutarSimulacao	Executa o núcleo da simulação, calcula métricas e atualiza a interface.
SimuladorJanelaProc	Callback principal da janela, responsável por inicialização e eventos de botões.
KosmosMain	Ponto de entrada do programa e inicialização da janela principal.

Table 5: Principais funções do simulador.

12 Validações Implementadas

O simulador possui algumas validações básicas para evitar execuções inválidas:

- impede iniciar a simulação sem processos carregados;
- impede iniciar a simulação com memória física menor ou igual a zero;
- garante ao menos um frame de RAM caso o valor informado seja menor que o tamanho de uma página;
- limita a quantidade de processos ao valor máximo definido por `MAX_PROCESSOS`;
- limita o histórico de execução ao valor definido por `MAX_INSTANTES`.

13 Limitações da Implementação Atual

Apesar de atender aos principais requisitos do projeto, a implementação atual possui algumas limitações importantes:

- O campo de memória virtual é exibido na interface, mas não influencia diretamente a simulação atual.
- A opção `Algoritmo Otimizado` está presente na interface, porém o código implementa uma substituição aleatória para essa opção.
- A simulação de memória acessa todas as páginas do processo a cada vez que ele executa, em vez de utilizar uma sequência realista de referências à memória.
- Não há persistência automática do relatório em arquivo externo; os resultados são exibidos na própria interface.
- A simulação utiliza um limite fixo de histórico de execução definido por `MAX_INSTANTES`.

14 Como Utilizar o Simulador

1. Abra o programa do simulador.
2. Clique no botão de procurar arquivo e selecione um CSV válido.
3. Confira se os processos foram carregados corretamente na tabela.
4. Informe o tamanho da memória física em MB.
5. Informe o tamanho da memória virtual, caso desejado para fins de configuração visual.
6. Defina o quantum para o algoritmo Round-Robin.
7. Escolha o algoritmo de escalonamento no ComboBox correspondente.
8. Escolha a política de substituição de páginas.
9. Clique no botão de iniciar para executar a simulação.
10. Analise o log, o gráfico de Gantt e as métricas finais exibidas na interface.

15 Conclusão

O simulador desenvolvido permite visualizar de maneira integrada conceitos essenciais de Sistemas Operacionais. A interface gráfica facilita a interação do usuário, enquanto os algoritmos implementados demonstram diferenças entre estratégias de escalonamento e políticas de gerenciamento de memória.

O projeto atende aos requisitos centrais da especificação ao implementar entrada por CSV, interface gráfica, três algoritmos de escalonamento, paginação, substituição de páginas, gráfico de Gantt e métricas de desempenho. Além disso, a documentação das limitações atuais permite identificar melhorias futuras, como a implementação real do algoritmo ótimo, o uso efetivo da memória virtual e a geração automática de relatórios em arquivo.